UC-3 — Enterprise Engineering & IT Copilot

LEXORA & NEXUS Solution Architecture and Build Specification · Version 1.0

Arnab Das · Feuji AI Practice · 7 October 2026

1. Executive summary

The Enterprise Engineering & IT Copilot gives all 100,000 employees one assistant for IT self-service, and gives the 15,000 engineers and 2,000 IT/SRE staff a code, incident and operations copilot inside the tools they already use. LEXORA builds nine model artefacts from the company's own tickets, code, runbooks and incidents; NEXUS serves them through one governed gateway with code-aware retrieval, permission-checked tools and human approval for every change to a production system.

Platform	Role in UC-3	What it delivers
LEXORA (offline)	Adapts models to the company's code, tickets, logs and vocabulary	Nine artefacts: ticket triage classifier, code embedding model, reranker, code completion (FIM) adapter, code chat and review adapter, text-to-query adapter, incident summariser, duplicate-ticket detector, change-risk model
NEXUS (online)	Runs the copilot for 100K users with security, routing and audit	IDE, chat, ITSM and alert surfaces; four-tier model router; hybrid code-and-document RAG; MCP tool layer; LangGraph workflows with approval gates; observability and FinOps

This use case differs from UC-1 (claims) and UC-2 (pharmacovigilance) in three ways that shape the design:

- **Latency extremes.** IDE code completion needs a response in about 300 ms; incident analysis may read 50,000 tokens of logs. One router must serve both.
- **Code is the sensitive asset.** The main risks are source-code and secret leakage, insecure generated code and licence contamination, rather than patient data.
- **Actions, not just answers.** Resetting access, restarting a pod or merging a fix changes real systems, so the tool and approval layer is central.

This document is written as a build specification. Section 9 catalogues every tool and library with its purpose, input, output and algorithm. Section 10 explains the algorithms. Section 11 defines the APIs, and section 12 gives the repository layout and a laptop-scale MVP stack for building the application.

2. Scope, personas and capabilities

Personas

Persona	Approximate population	Main surface	Top needs
General employee	83,000	Teams/Slack bot, IT portal	Password and MFA help, access requests, software installs, VPN and laptop issues, "how do I" policy questions

Persona	Approximate population	Main surface	Top needs
Software engineer	15,000	IDE plugin, chat, pull requests	Code completion, codebase Q&A, code explanation, PR review, test generation, internal API usage
SRE / platform engineer	1,200	Chat, incident channel, terminal	Alert triage, log and trace analysis, runbook execution, change risk, postmortems
IT service-desk agent	800	ServiceNow/Jira Service Management	Ticket triage and routing, suggested resolutions, duplicate detection, knowledge-article drafting

Capabilities

ID	Capability	Who	Primary LEXORA artefact	NEXUS path
C1	IT self-service resolution (password, MFA, VPN, software)	Employees	L1 triage classifier	Helpdesk workflow + Okta/Intune/ServiceNow tools
C2	Ticket triage, routing and duplicate detection	Service desk	L1 classifier, L8 duplicate detector	Event workflow on ticket creation
C3	Enterprise knowledge Q&A (Confluence, SharePoint, KB, runbooks)	Everyone	L2 embeddings, L3 reranker	Hybrid RAG + T2 general model
C4	Code completion in the IDE	Engineers	L4 FIM completion adapter	Dedicated low-latency completion pool
C5	Codebase Q&A, explanation and test generation	Engineers	L2, L3, L5 code chat adapter	Code RAG with code graph + T1b
C6	Pull-request review	Engineers	L5 code chat and review adapter	PR webhook workflow + SAST validators
C7	Text-to-query (SQL, PromQL, LogQL, KQL)	Engineers, SRE, analysts	L6 text-to-query adapter	Schema RAG + execution sandbox
C8	Incident copilot: summarise, correlate, suggest runbook, draft postmortem	SRE	L7 incident summariser	Incident workflow + observability tools + approval gate
C9	Change-risk scoring and CAB brief	SRE, change managers	L9 change-risk model	Change workflow on ServiceNow change request
C10	Access request agent with manager approval	Employees, managers	L1 classifier	IAM workflow + approval token
C11	IaC and Kubernetes config generation and review	Platform engineers	L5 adapter	Policy validators (Checkov, Conftest)

Service levels and success KPIs

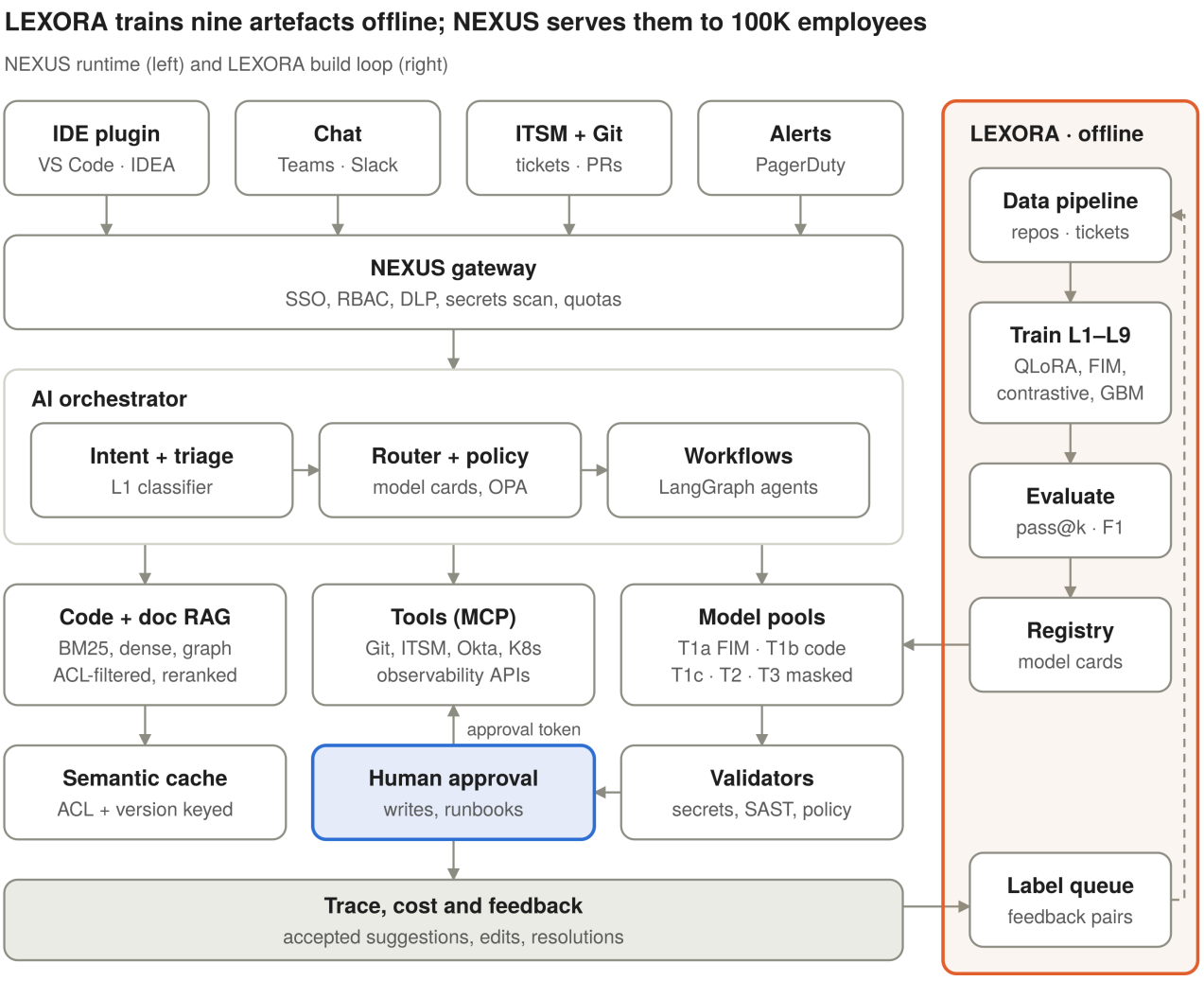
Request type	Mode	Target latency (p95)	KPI it moves
IDE inline completion	Sync, streamed	≤ 300 ms first token, ≤ 600 ms complete	Suggestion acceptance rate ≥ 25%

Request type	Mode	Target latency (p95)	KPI it moves
Code or knowledge chat	Sync, streamed	≤ 2 s first token	Weekly active engineers; answer helpfulness ≥ 4/5
IT self-service conversation	Sync, streamed	≤ 2 s first token	Ticket deflection ≥ 30% of L1 volume
Ticket triage on creation	Async event	≤ 10 s	Correct first assignment ≥ 85%; reassignment rate down 40%
PR review comment set	Async event	≤ 3 minutes after PR opens	Review comments accepted ≥ 35%; time to first review down 50%
Incident brief on alert	Async workflow	≤ 2 minutes after page	MTTA down 30%; MTTR down 20%
Change-risk score	Async event	≤ 1 minute	Change-failure rate down 15%

KPI values are design targets to confirm against a baseline in the first month.

3. Reference architecture

NEXUS (left in the diagram) runs the copilot; LEXORA (right) is the offline build loop that produces the models NEXUS routes to and learns from NEXUS's feedback.



UC-3 architecture · NEXUS runtime and LEXORA build loop

Every request enters through the gateway, where identity, data classification and a secrets scan run before anything reaches a model. The orchestrator classifies intent, the router chooses a model tier from LEXORA's registry cards, retrieval applies repository and space permissions before ranking, and any tool that changes a system needs a human approval token. Every step is traced; human edits, accepted suggestions and resolved tickets flow back to LEXORA as training and evaluation data.

4. LEXORA role: what is trained and why

4.1 Adaptation decision per task

LEXORA trains a model only where behaviour or vocabulary must change. Facts that change daily (documentation, code, tickets) stay in NEXUS retrieval.

Task	Changes often?	Decision	Why
Classify tickets to category, assignment group, priority	Groups change quarterly	Fine-tune encoder classifier (L1)	High volume, needs ms latency and calibrated confidence; labels already exist in ITSM history

Task	Changes often?	Decision	Why
Find the right code, doc or runbook	Content changes daily	RAG, with fine-tuned embeddings (L2) and reranker (L3)	Generic embedders miss internal service names, acronyms and code idioms
Complete code inline	Code changes daily, style is stable	FIM adapter (L4) + repo context from RAG	Learns internal frameworks and conventions; context supplies current code
Answer code questions, review PRs	Code changes daily	Adapter (L5) + code RAG	Learns review style and internal libraries; RAG supplies the actual files
Write SQL, PromQL, LogQL, KQL	Schemas change monthly	Adapter (L6) + schema RAG + execution check	Query dialects and internal metric names need tuning; schemas retrieved at run time
Summarise incidents and propose causes	Systems change	Adapter (L7) with DPO + RAG over postmortems	House postmortem style and SRE judgement; past incidents retrieved
Detect duplicate tickets	Continuous	Embedding similarity + learned threshold (L8)	Cheap, explainable, no generation needed
Score change risk	Continuous	Gradient-boosted model (L9)	Tabular features (blast radius, history, timing); explainable with SHAP
Know current on-call, owners, CMDB facts	Continuous	Tool call	Never trusted to model memory

4.2 The nine LEXORA artefacts

ID	Artefact	Base / algorithm	Training data	Input at inference	Output
L1	Ticket triage classifier	ModernBERT or DeBERTa-v3 encoder, multi-head classification; temperature-scaled	200K–1M historical tickets with final category, group, priority	Ticket title + description + requester metadata	Category, assignment group, priority, each with calibrated probability
L2	Code and document embedding model	bge-m3 or a code embedder (e.g. jina-embeddings-v2-base-code, nomic-embed-code) fine-tuned with contrastive loss	(query, relevant chunk) pairs mined from PR descriptions ↔ diffs, ticket ↔ resolving KB article, question ↔ accepted answer	Text or code chunk	Dense vector (768–1,024 dims)
L3	Reranker	Cross-encoder (bge-reranker-v2-m3) fine-tuned on clicks and accepted answers	(query, passage, relevance) triples with hard negatives	Query + candidate passage	Relevance score
L4	Code completion (FIM) adapter	Small code model (1.5–7B, Qwen-Coder or StarCoder2 family) + continued pretraining with fill-in-the-middle + LoRA	De-duplicated, secret-scrubbed, licence-filtered internal repositories	Prefix + suffix + retrieved repo snippets	Middle code span

ID	Artefact	Base / algorithm	Training data	Input at inference	Output
L5	Code chat and review adapter	Code instruct model (≈ 30B class) + QLoRA SFT, then DPO on reviewer preferences	Historical PR review threads (comment → fix accepted), internal Q&A, style guides	Question or diff + retrieved code context	Answer, or review comments anchored to file and line
L6	Text-to-query adapter	7–8B code or instruct model + QLoRA	(question, schema, query) pairs from query logs; synthetic pairs filtered by execution	Question + retrieved schema / metric catalogue	SQL / PromQL / LogQL / KQL query
L7	Incident summariser	7–8B instruct model + QLoRA SFT, then DPO with SRE rankings	Past incidents: alerts, timelines, chat logs, postmortems	Alert + log templates + metrics summary + deploys + similar incidents	Incident brief: impact, timeline, ranked hypotheses, suggested runbook
L8	Duplicate-ticket detector	L2 embeddings + HNSW nearest neighbours + logistic threshold	Ticket pairs marked duplicate by agents	New ticket	Top-k candidate duplicates with probability
L9	Change-risk model	LightGBM gradient-boosted trees + SHAP explanations	Historical changes with outcome (success, rollback, incident)	Change features (services touched, size, test coverage, author history, time window, dependency fan-out)	Risk probability + top contributing factors

4.3 Data pipeline for engineering data

Engineering data needs three extra controls beyond the UC-1/UC-2 pipeline: secret removal, licence filtering and structure-aware chunking of code.

1. **Collect** — Git repositories (GitHub/GitLab/Bitbucket APIs), ITSM tickets (ServiceNow/Jira), Confluence/SharePoint, incident records (PagerDuty, postmortem docs), query logs (warehouse, Prometheus, Loki), change records, CMDB.
2. **Secret scan and redact** — gitleaks / TruffleHog / detect-secrets scan every file and commit; matches are replaced with typed placeholders (`<AWS_KEY>`). Files with unresolved findings are excluded.
3. **PII redaction** — Presidio removes employee names, emails and phone numbers from tickets and chat logs (requester IDs kept as hashed surrogates).
4. **Licence filter** — ScanCode Toolkit identifies third-party code and licences; vendored and copyleft code is excluded from FIM training.
5. **Deduplicate** — MinHash LSH (datasketch) removes near-duplicate files, generated code, lock files and minified bundles.
6. **Parse and chunk** — tree-sitter splits code by function and class; Docling/Unstructured parses documents by heading; Drain3 converts raw logs into templates.
7. **Label and pair** — ticket outcomes, PR ↔ diff pairs, review threads, accepted answers and incident timelines become supervised pairs.
8. **Register** — immutable dataset versions with source mix, scan reports and licence summary in the dataset registry.

4.4 Training recipes

Artefact	Recipe	Key hyperparameters	Library
L1 triage	Full fine-tune of encoder with three classification heads; class-weighted cross-entropy; temperature scaling on validation set	LR 2e-5, batch 32, 3 epochs, max length 512	Hugging Face Transformers, scikit-learn (calibration)
L2 embeddings	Contrastive fine-tune with in-batch negatives plus mined hard negatives	MultipleNegativesRankingLoss, temperature 0.05, batch 64–256, 1–2 epochs	sentence-transformers
L3 reranker	Cross-encoder fine-tune, binary cross-entropy on (query, positive, hard negatives)	LR 2e-5, 1–2 epochs, 1:4 positive:negative	sentence-transformers (CrossEncoder) or FlagEmbedding
L4 FIM	Continued pretraining with FIM transformation (50% of samples, PSM and SPM formats), then LoRA	FIM rate 0.5, LoRA r=16–32, LR 1e-4, context 4–8K	Transformers + PEFT; Axolotl or Unsloth for speed
L5 code chat/review	QLoRA SFT on review threads and Q&A, then DPO on (accepted, rejected) review comments	r=16, alpha=32, NF4; DPO beta 0.1	PEFT, TRL (SFTTrainer, DPOTrainer), bitsandbytes
L6 text-to-query	QLoRA SFT; synthetic pairs from a teacher model kept only if the query executes and returns the expected shape	r=16, 3 epochs	PEFT, TRL, sqlglot (parse/validate)
L7 incident	QLoRA SFT on incident → postmortem summary, then DPO with SRE preference pairs	r=16, DPO beta 0.1	PEFT, TRL
L8 duplicates	Fit logistic regression on cosine similarity + metadata (same requester, same CI, time gap)	Threshold chosen at precision ≥ 0.9	scikit-learn, FAISS/hnswlib
L9 change risk	LightGBM binary classifier, time-based split, SHAP for explanation	num_leaves 31–63, early stopping, class weights	LightGBM, SHAP, Optuna (tuning)

4.5 Evaluation and release gates

Artefact	Primary metric	Release gate	Other checks
L1 triage	Top-1 assignment-group accuracy; macro-F1 on category	≥ 0.85 accuracy; ECE ≤ 0.05	Per-group recall so small teams are not starved
L2 + L3 retrieval	Recall@10, MRR, nDCG@10 on internal query set	Recall@10 ≥ 0.85; nDCG@10 +10% over base embedder	Latency per query; index size
L4 FIM	Exact match and pass@1 on held-out repo infilling tasks; online acceptance rate	Beats base model by ≥ 5 points pass@1; no secret regurgitation	Latency p95 ≤ 300 ms first token
L5 code chat/review	pass@1 on internal unit-test tasks; reviewer acceptance of comments; SWE-bench-style issue tasks on internal repos	Acceptance ≥ 35% in shadow; no rise in SAST findings in suggested code	Hallucinated API rate (calls to functions that do not exist) ≤ 2%

Artefact	Primary metric	Release gate	Other checks
L6 text-to-query	Execution accuracy (result matches gold)	≥ 0.80 on internal set; 100% read-only statements	Dialect validity via sqlglot / promtool
L7 incident	Faithfulness of brief to source signals; SRE rubric score	Unsupported-claim rate ≤ 2%; rubric ≥ 4/5	Correct-runbook-in-top-3 rate
L8 duplicates	Precision@1 at operating threshold	Precision ≥ 0.90	Recall tracked
L9 change risk	AUC-ROC, precision in top-decile risk	AUC ≥ 0.75	SHAP sanity review with change managers

All artefacts register in MLflow with the model-card contract defined in the LEXORA v2 document (section 10). That covers task tags, eval scorecard, latency and cost profile, the data classes permitted, known failure modes and a rollback target.

5. NEXUS role: capacity model for 100,000 employees

Code completion, not chat, dominates GPU demand. It produces hundreds of small, latency-critical requests per second, so it gets its own pool and is sized separately. All inputs are planning assumptions to replace with telemetry.

5.1 Demand

Workload	Assumption	Peak load
IDE completion	15,000 engineers × 50% coding at peak × 3 requests/min after debounce and cancellation	≈ 375 req/s; ~1,500 prompt tokens (≈ 70% prefix-cache hit) / ~40 output tokens
Code chat and Q&A	7,500 active engineers × 4 chats/hour	≈ 8.3 req/s; ~6,000 / 500 tokens
PR review	~3,000 PRs/day, peaks at 3× average	≈ 0.1 req/s; ~20,000 / 1,500 tokens
IT self-service and knowledge Q&A	25,000 daily active × 30% in peak hour × 2 requests/hour	≈ 4.2 req/s; ~3,000 / 350 tokens
Ticket triage	~10,000 tickets/business day	≈ 0.35/s on encoder (milliseconds each)
Incident briefs	Tens per day, bursty during major incidents	Low rate, up to 50,000-token contexts
Chat total with 2× burst, after 20% cache hits	(8.3 + 4.2 + 0.1) × 2 × 0.8	≈ 20 req/s reach a generative model

5.2 Model tiers and GPU plan

Tier	Model	Serves	Hosting	Planning GPUs (80 GB class)
T0	L1 triage classifier, L2 embedder, L3 reranker, L8 duplicates	Classification, retrieval, dedupe	TEI / ONNX Runtime; GPU or CPU	2 (plus 2 burst for re-indexing)
T1a	L4 FIM completion adapter on 1.5–7B code model	IDE completion	vLLM, dedicated pool, prefix caching, speculative decoding	6–8
T1b	L5 code chat/review adapter on ≈ 30B code model (FP8)	Code chat, PR review, IaC review	vLLM multi-LoRA	6

Tier	Model	Serves	Hosting	Planning GPUs (80 GB class)
T1c	L6 text-to-query + L7 incident adapters on 8B base	Queries, incident briefs	vLLM multi-LoRA	2
T2	General instruct 8–30B	IT self-service, knowledge Q&A	vLLM	2–3
T3	Frontier model via private endpoint	Hard debugging, architecture questions, agent planning	Managed, zero data retention, code-sharing policy enforced	Managed (per token)
Total self-hosted				**≈ 20–23**

5.3 What the capacity model tells the architecture

- **Isolate the completion pool.** A single long incident prompt must never delay keystroke-level completions. The completion pool also has its own autoscaling on queue depth.
- **Client-side debounce and cancellation are part of the design.** The IDE plugin waits about 150 ms after typing stops and cancels in-flight requests on the next keystroke, roughly halving load.
- **Prefix caching is the biggest single saving for completion.** The file header and imports repeat across keystrokes.
- **Re-indexing is a batch workload.** Embedding a large monorepo after a big merge runs on separate burst capacity, never on the interactive T0 pool.

6. NEXUS role: routing, retrieval and code intelligence

6.1 Router policy

Request	Route	Fallback
Inline completion	T1a only (fixed route, no scoring, latency budget too small)	Return no suggestion; never block typing
Code question on a repo the user can read	Code RAG → T1b	T3 if repo classification allows external models and question is complex
Code from a “restricted” repo (e.g. security, payments)	T1b self-hosted only	No external fallback, by policy gate
PR review	T1b + validators	Queue and retry; skip review rather than degrade silently
IT self-service	T0 intent → workflow → T2	Hand off to human service desk with conversation summary
Knowledge question	Doc RAG → T2	T3 for multi-document synthesis
Text-to-query	Schema RAG → T1c (L6) → validate → execute read-only	Return query unexecuted with warning
Incident brief	Incident workflow → T1c (L7)	T3 with log redaction if context exceeds self-hosted window

The scoring formula, cascade rules and router evaluation follow the NEXUS v2 document (section 5), with one addition: **repository classification** (public, internal, confidential, restricted) is a hard input to the policy gate, alongside data classification.

6.2 Knowledge and code corpora

Corpus	Source system	Chunking	Freshness	ACL source
Source code	GitHub / GitLab / Bitbucket	tree-sitter function/class chunks with file path, symbol name, imports	Webhook on push; incremental re-embed of changed files	Repository permissions
Code graph	SCIP indexers / universal-ctags / tree-sitter	Symbols, definitions, references, call edges	On merge to default branch	Repository permissions
API specs	OpenAPI / gRPC proto / GraphQL schema files	One chunk per endpoint or message	On merge	Repository permissions
Architecture docs, ADRs, runbooks	Confluence, SharePoint, Git markdown	Heading-aware, 300–800 tokens with overlap	Webhook / hourly crawl	Space and page permissions
KB articles	ServiceNow / Jira Service Management	Whole article plus question variants	On publish	Audience (employee, IT, engineering)
Resolved tickets	ITSM	Problem + resolution summary	Nightly	Assignment group visibility
Incidents and postmortems	PagerDuty, postmortem repo	Timeline, root cause, actions as separate chunks	On close	Engineering-wide or restricted
Data schemas and metric catalogues	Warehouse information schema, dbt docs, Prometheus metadata	One chunk per table or metric family	Nightly	Data-access entitlements

6.3 Retrieval pipeline for code questions

1. **Understand** — extract repository, service, file and symbol mentions; detect language; rewrite into lexical and semantic queries.
2. **Scope** — restrict to repositories the user can read and, if mentioned, the named service.
3. **Retrieve in parallel** — BM25 for identifiers and error strings; dense search with L2 embeddings; code-graph expansion (definition → references → callers, one or two hops).
4. **Fuse and rerank** — reciprocal rank fusion, then L3 cross-encoder to the top 8–15 chunks within a token budget.
5. **Assemble** — include file paths, line ranges and commit SHA so answers cite exact locations.
6. **Validate** — referenced functions and files must exist in the index (hallucinated-API check); citations must resolve to the retrieved chunks.

6.4 Completion context assembly (IDE)

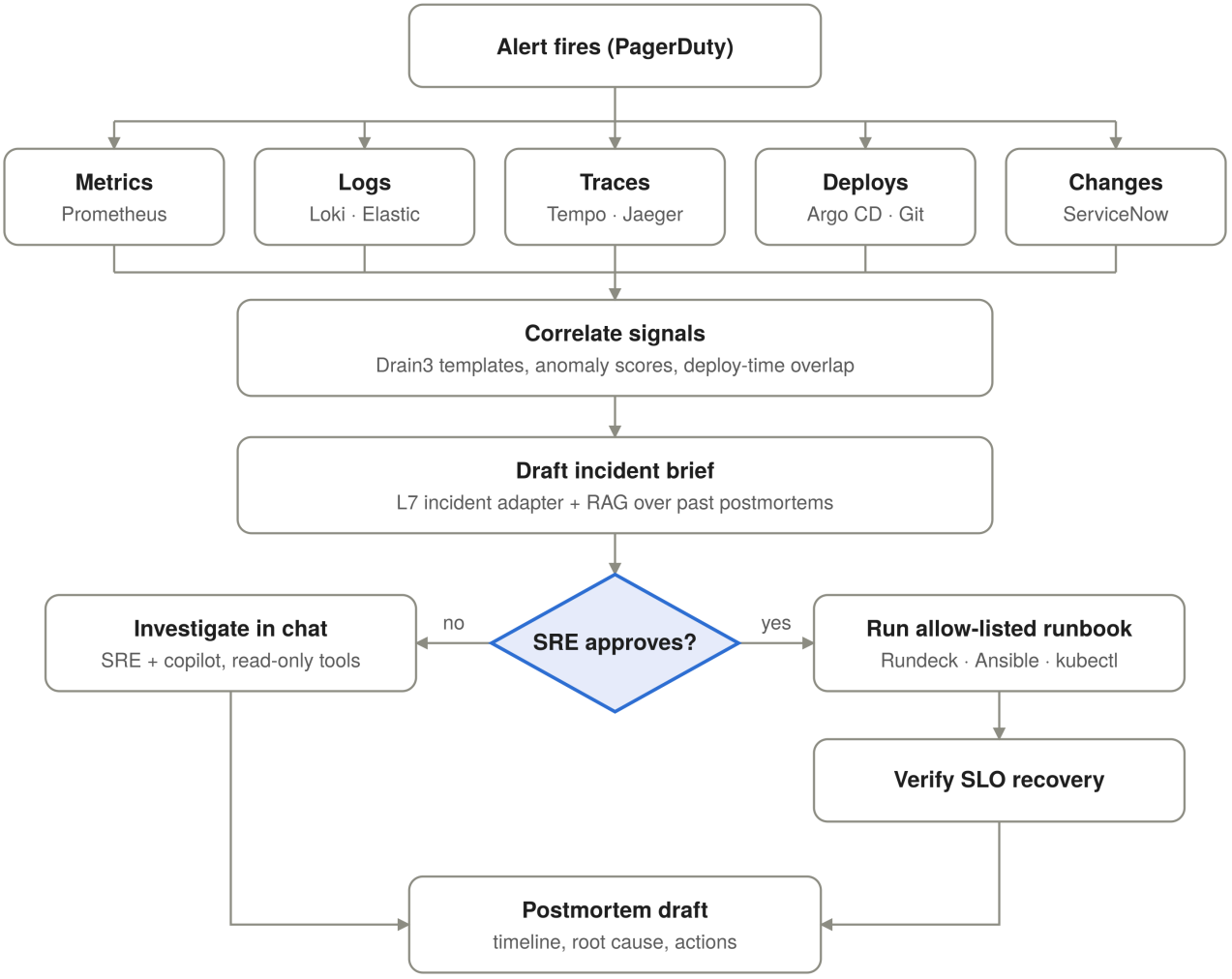
The plugin sends the prefix, suffix, file path and language. NEXUS adds up to about 1,000 tokens of context: definitions of symbols near the cursor (code graph), recently edited open files, and the top similar snippets from the same repository (L2). The FIM prompt is built as prefix–suffix–middle and sent to T1a. Post-filters drop suggestions that contain secrets, repeat the prefix, or fail a quick syntax check with tree-sitter.

7. NEXUS role: agent workflows and tools

7.1 Incident copilot workflow

The copilot gathers and reasons; an SRE approves every action

UC-3 incident copilot workflow (LangGraph, checkpointed)



Incident copilot · gather, correlate, draft, approve, act

Step	What runs	Tool / library	Input	Output
1. Alert received	Event handler	PagerDuty / Alertmanager webhook	Alert payload	Incident context object, start time
2. Gather metrics	Tool	Prometheus HTTP API (PromQL)	Service, time window	Anomalous series with baselines
3. Gather logs	Tool + Drain3	Loki / Elasticsearch / Splunk APIs	Service, window	Log templates with counts, new templates since baseline
4. Gather traces	Tool	Tempo / Jaeger APIs	Service, window	Slowest spans, error spans, dependency hot spots

Step	What runs	Tool / library	Input	Output
5. Gather changes	Tool	Argo CD, GitHub/GitLab, ServiceNow change API	Service, window	Deploys, config changes, feature-flag flips
6. Correlate	Rules + scoring	Isolation Forest / STL residuals; time-overlap scoring	Signals from steps 2–5	Ranked candidate causes with evidence
7. Similar incidents	RAG	L2 + L3 over postmortems	Incident signature	Top-3 similar past incidents and their fixes
8. Draft brief	T1c (L7)	vLLM	All evidence	Impact, timeline, hypotheses, suggested runbook
9. **Human gate**	SRE	Slack/Teams interactive message	Brief + proposed action	Approve, modify or reject
10. Execute	Tool	Rundeck / Ansible / kubectl via allow-listed MCP tool	Approved action + approval token	Execution log
11. Verify	Tool	Prometheus SLO query	SLO target	Recovered or not; loop back to step 6 if not
12. Postmortem draft	T1c (L7)	Confluence/Git write (draft only)	Full trace of incident	Postmortem draft for review

7.2 IT self-service workflow (password / MFA / access)

1. **Classify intent** with L1 (password reset, MFA reset, access request, software install, VPN, hardware, other).
2. **Verify identity** step-up (Okta Verify push or SSO re-authentication) for any account action.
3. **Resolve or act**:
   - Password/MFA → Okta API.
   - Software → Intune/Jamf catalogue.
   - Access → ServiceNow request with manager approval.
4. **Escalate** when confidence is low or the user asks: create a ticket pre-filled by L1 with the conversation summary.
5. **Close the loop**: resolution outcome and satisfaction score feed L1 retraining and deflection metrics.

7.3 PR review workflow

1. A webhook on PR open or update triggers the workflow.
2. Fetch the diff and changed files, and retrieve related code (callers and callees) and team style guides.
3. T1b (L5) produces review comments as structured JSON, each with file, line, severity, category and suggestion.
4. Validators run: Semgrep (SAST), the language linter, secret scan of suggested code, and a licence snippet match.
5. Post the comments to the PR as a bot review. Comments are never auto-approved, and the bot never auto-merges.
6. Accepted and dismissed comments are logged as DPO preference pairs for L5.

7.4 Tool catalogue (MCP servers)

Tool	System	Read / write	Approval required	Typical use
git.search, git.read_file, git.diff	GitHub / GitLab	Read	No	Code RAG, PR review
git.comment_pr	GitHub / GitLab	Write (comment only)	No (bot identity, non-blocking)	PR review
git.open_pr	GitHub / GitLab	Write	Yes, author confirmation	Generated fix or IaC change
itsm.create_ticket, itsm.update_ticket	ServiceNow / Jira SM	Write	No for create; yes for closing on user's behalf	Helpdesk, triage
itsm.request_access	ServiceNow	Write	Manager approval in ServiceNow	Access requests
idp.reset_password, idp.reset_mfa	Okta / Entra ID	Write	Step-up identity verification	Self-service
mdm.install_app	Intune / Jamf	Write	No (catalogue apps only)	Software requests
obs.query_metrics / logs / traces	Prometheus, Loki/Elastic/Splunk, Tempo/Jaeger	Read	No	Incident copilot, text-to-query
deploy.list_changes	Argo CD, CI system	Read	No	Incident correlation, change risk
runbook.execute	Rundeck / Ansible / kubectl (allow-listed verbs)	Write	Yes, SRE approval token bound to action	Remediation
kb.search, kb.draft_article	Confluence / ServiceNow KB	Read / write draft	Publication needs owner approval	Knowledge
oncall.who	PagerDuty	Read	No	Routing and escalation
cmdb.lookup	ServiceNow CMDB	Read	No	Service ownership, dependencies
db.query_readonly	Warehouse / replicas	Read	No (read-only role, row limits, timeout)	Text-to-query

Write tools check an approval token bound to the exact action and parameters the approver saw. Every tool has a timeout, retry budget and circuit breaker, and writes carry idempotency keys.

8. NEXUS role: security, guardrails and operations

8.1 Engineering-specific threats and controls

Threat	Control	Tooling
Secrets pasted into prompts or present in retrieved code	Gateway secret scan; redaction before any model call; block on high-confidence keys	detect-secrets / gitleaks rules in gateway; LLM Guard secrets scanner
Source code sent to an external model	Repository classification as router hard gate; restricted repos never leave self-hosted tiers	OPA policy; repo-classification service

Threat	Control	Tooling
Prompt injection via code comments, issues, logs or web pages	Untrusted-context separation; retrieved content never treated as instructions; injection classifier on inputs and retrieved chunks; tool allow-lists per workflow step	Prompt Guard / LLM Guard; NeMo Guardrails
Insecure generated code	SAST on every suggested diff and review fix	Semgrep, CodeQL (scheduled), Bandit for Python
Insecure IaC	Policy checks before suggesting or opening PRs	Checkov, tfsec/Trivy, Conftest (OPA), kubeconform
Licence contamination	Snippet-match generated code against known open-source corpora above a length threshold	ScanCode, internal snippet index
Over-privileged agent actions	Least-privilege service identities; allow-listed verbs; approval tokens; dry-run first	MCP tool schemas, OPA, Rundeck ACLs
Data exfiltration through text-to-query	Read-only roles, row limits, column masking, query timeouts	Database roles, sqlglot static checks
Code execution risk (tests, generated scripts)	Sandboxed execution with no network and short timeouts	gVisor or Firecracker microVMs; Kubernetes jobs

8.2 Caching

Cache	Key	Notes
Completion prefix (KV) cache	Token prefix (in vLLM)	Largest saving for T1a
Embedding cache	File path + content hash + embedder version	Only changed chunks re-embed on push
Retrieval cache	Query + repo scope + ACL group hash + index version	Invalidated on index update
Semantic answer cache	Query embedding + ACL group hash + corpus version + model version	Only for knowledge and IT self-service answers; never for code from restricted repos

8.3 Observability and KPIs

Plane	Metrics
Completion	Requests/s, p50/p95 latency, cancellation rate, acceptance rate, characters accepted per user
Chat and RAG	TTFT, Recall@10 on canary questions, citation validity, hallucinated-API rate, helpfulness votes
Workflows	Deflection rate, triage accuracy, reassignment rate, PR comment acceptance, incident brief time, approval wait time
Safety	Secrets blocked, injection detections, SAST findings in suggestions, policy-gate denials
Cost	GPU-seconds by tier, tokens by team, cost per resolved ticket, cost per accepted suggestion

Tracing uses OpenTelemetry end to end, with Langfuse for prompt, retrieval and generation spans. The LLMOps loop and FinOps model are those of the NEXUS v2 document (section 10).

9. Master catalogue: tools, libraries, purpose, input, output, algorithm

The tables below list every component needed to build UC-3. Versions should be pinned at build time, and open-source licences checked before production use.

9.1 Data ingestion and preparation (LEXORA)

Tool / library	Purpose	Input	Output	Core algorithm
PyGithub / python-gitlab	Pull repos, PRs, review threads	Org, repo, date range	Files, diffs, comments	REST/GraphQL pagination
ServiceNow / Jira REST APIs (requests, atlassian-python-api)	Pull tickets, KB, changes	Query filters	Ticket JSON	REST pagination
Docling / Unstructured	Parse PDF, DOCX, HTML, Confluence exports	Documents	Structured elements (headings, tables, text)	Layout analysis, table structure recognition
tree-sitter (py-tree-sitter + language grammars)	Parse code into syntax trees; chunk by function/class	Source files	AST nodes, chunks with symbol names	Incremental GLR parsing
gitleaks / TruffleHog / detect-secrets	Find secrets in code and history	Files, commits	Findings with type and location	Regex + entropy detection; verified-credential checks (TruffleHog)
Microsoft Presidio	Detect and anonymise PII in tickets and chats	Text	Redacted text + entity map	NER (spaCy/transformers) + pattern recognisers
ScanCode Toolkit	Detect licences and copyright in code	Files	Licence findings per file	Licence text matching, rule-based detection
datasketch	Near-duplicate detection	Token shingles	Duplicate clusters	MinHash + locality-sensitive hashing
Drain3	Turn raw logs into templates	Log lines	Template IDs, parameters, counts	Fixed-depth parse tree (Drain)
pandas / Polars / PyArrow	Tabular processing, Parquet storage	Raw records	Clean datasets	Columnar processing
DVC or Delta Lake / Iceberg tables	Dataset versioning	Dataset files	Versioned, reproducible snapshots	Content-addressed storage / table snapshots
Label Studio / Argilla	Human labelling and review of pairs and preferences	Samples	Labels, preference pairs	Annotation workflows, agreement scoring

9.2 Model training and evaluation (LEXORA)

Tool / library	Purpose	Input	Output	Core algorithm
PyTorch	Deep-learning runtime	Tensors	Trained weights	Autograd, mixed precision
Hugging Face Transformers	Model loading and training loops	Model ID, dataset	Checkpoints	Transformer architectures
Hugging Face Datasets	Dataset loading, mapping, streaming	Parquet/JSONL	Tokenised datasets	Memory-mapped Arrow
PEFT	LoRA / QLoRA adapters	Base model, config	Adapter weights	Low-rank update W + BA

Tool / library	Purpose	Input	Output	Core algorithm
bitsandbytes	4-bit/8-bit quantisation for QLoRA	Base weights	NF4 quantised weights	NormalFloat4 + double quantisation
TRL	SFT, DPO, reward modelling	Instruction / preference data	Tuned adapters	Supervised fine-tuning; Direct Preference Optimisation
Axolotl / Unsloth	Config-driven, faster fine-tuning (incl. FIM continued pretraining)	YAML config, data	Checkpoints	Fused kernels, packing, gradient checkpointing
DeepSpeed / FSDP (Accelerate)	Multi-GPU training for larger code models	Model, data	Sharded training	ZeRO / fully sharded data parallel
sentence-transformers	Fine-tune embedders and cross-encoders (L2, L3)	Pairs / triples	Embedding and reranker models	Contrastive learning (MNRL / InfoNCE); cross-encoder BCE
FlagEmbedding	bge-m3 and bge-reranker training utilities	Pairs with hard negatives	Fine-tuned BGE models	Contrastive + hard-negative mining
scikit-learn	Calibration, logistic threshold, metrics	Predictions, labels	Calibrated models, scores	Temperature/Platt scaling, logistic regression
LightGBM	Change-risk model (L9)	Tabular change features	Risk probability	Gradient-boosted decision trees (histogram-based, leaf-wise)
SHAP	Explain L9 predictions	Model + features	Per-feature contributions	Shapley values (TreeSHAP)
Optuna	Hyperparameter search	Search space, objective	Best parameters	Tree-structured Parzen Estimator
PyOD / scikit-learn IsolationForest; statsmodels STL	Anomaly scoring for incident correlation	Metric series, log-template counts	Anomaly scores	Isolation Forest; seasonal-trend decomposition
sqlglot / promtool	Validate generated queries	SQL / PromQL text	Parse result, dialect errors	SQL parser/transpiler; PromQL parser
Evaluation harness (custom, HumanEval-style)	pass@k on internal unit-test tasks	Generated code + tests	pass@k	Sandboxed test execution
MLflow	Experiment tracking, model registry, model cards	Runs, artefacts	Registered model versions	Tracking server + registry
Weights & Biases (optional)	Experiment dashboards	Metrics	Charts, sweeps	—

9.3 Serving and inference (shared)

Tool / library	Purpose	Input	Output	Core algorithm
vLLM	Serve generative models: FIM, chat, multi-LoRA, structured output	Prompts, adapter name	Tokens (streamed)	PagedAttention, continuous batching, prefix caching, speculative decoding
Text Embeddings Inference (TEI)	Serve embedders and rerankers	Text batches	Vectors / scores	Dynamic batching, Flash Attention

Tool / library	Purpose	Input	Output	Core algorithm
ONNX Runtime / NVIDIA Triton	Serve L1 encoder classifier at low latency	Ticket text	Class probabilities	Graph optimisation, INT8 quantisation
xgrammar / Outlines (via vLLM)	Constrain output to JSON schema or grammar	Schema, logits	Valid structured output	Grammar-constrained decoding
AWQ / GPTQ / FP8 (llm-compressor)	Quantise models for serving	Weights	Quantised weights	Activation-aware / second-order weight quantisation
Ollama or llama.cpp (dev only)	Run small models on a laptop for the MVP	GGUF model	Tokens	CPU/GPU quantised inference

9.4 Retrieval and knowledge (NEXUS)

Tool / library	Purpose	Input	Output	Core algorithm
OpenSearch / Elasticsearch	Lexical search over code and docs; also vectors	Queries, filters	Ranked hits	BM25; HNSW for k-NN
Qdrant / Milvus / pgvector	Vector store with payload filters	Query vector + ACL filter	Nearest chunks	HNSW approximate nearest neighbour
FAISS / hnswlib	In-process ANN for duplicates (L8) and experiments	Vectors	Neighbours	IVF / HNSW
Neo4j (+ networkx for analysis)	Code graph and service dependency graph	Symbols, edges	Graph traversals	Property-graph traversal; k-hop expansion
SCIP indexers / universal-ctags	Build precise symbol definitions and references	Repositories	Symbol index	Language-specific semantic indexing
LlamaIndex or custom pipeline	Retrieval orchestration helpers	Queries	Context bundles	Hybrid retrieval, RRF
RAGAS / DeepEval	Evaluate retrieval and grounded generation	Q&A sets, contexts, answers	Faithfulness, context precision/recall	LLM-assisted and embedding-based metrics

9.5 Orchestration, gateway and tools (NEXUS)

Tool / library	Purpose	Input	Output	Core algorithm
FastAPI + Pydantic	Service APIs and schemas	HTTP requests	JSON / SSE streams	Async request handling, schema validation
LangGraph	Stateful workflows with checkpoints and human-in-the-loop	Graph definition, state	Workflow results	State machine / graph execution with persistence
LiteLLM (or custom router)	Unified model API, routing, fallbacks, budgets	Requests	Model responses	Policy- and score-based routing
Kong / Envoy / cloud API gateway	Edge gateway: auth, rate limits, WAF	HTTP	Authenticated requests	Token bucket rate limiting
Open Policy Agent (OPA)	Policy gate: repo classification, roles, tool permissions	Request attributes	Allow / deny + reasons	Rego policy evaluation

Tool / library	Purpose	Input	Output	Core algorithm
MCP Python SDK / FastMCP	Expose tools to agents with typed schemas	Tool calls	Tool results	JSON-RPC over stdio/HTTP
Kafka / Redis Streams + Celery or Temporal	Event intake and async jobs (webhooks, re-indexing)	Events	Processed jobs	Durable queues; workflow retries
Redis	Exact and semantic caches, session state	Keys, vectors	Cached values	Key-value + vector similarity
PostgreSQL	Workflow checkpoints, audit metadata, app data	Records	Durable state	Relational storage
Slack Bolt / Microsoft Bot Framework (Teams)	Chat surfaces and interactive approvals	Messages, button actions	Bot replies	Event subscriptions
VS Code extension API (TypeScript); JetBrains Platform SDK (Kotlin)	IDE plugin for completion and chat	Editor state	Inline suggestions, chat panel	Debounce, cancellation, streaming
Continue.dev / TabbyML (reference)	Open-source IDE assistant references to study or extend	—	—	—

9.6 Guardrails, security and code quality (NEXUS)

Tool / library	Purpose	Input	Output	Core algorithm
LLM Guard	Input/output scanners: secrets, PII, injection, toxicity	Prompt / response	Pass/fail + sanitised text	Classifiers + regex + entropy
Llama Prompt Guard / similar classifier	Detect prompt injection and jailbreaks	Text	Injection score	Fine-tuned transformer classifier
NeMo Guardrails	Conversation rails and topic restrictions	Dialogue	Allowed / redirected flows	Colang-defined rails
Semgrep	SAST on suggested code and PR fixes	Code	Findings	Pattern-based semantic matching
CodeQL (scheduled)	Deep static analysis	Repository	Vulnerability alerts	Dataflow / taint analysis over code database
Checkov / Trivy / tfsec / Conftest / kubeconform	IaC and Kubernetes policy validation	Terraform, YAML, Dockerfiles	Policy violations	Rule and OPA policy evaluation; schema validation
gVisor / Firecracker	Sandbox for executing tests and generated scripts	Code + tests	Results, logs	User-space kernel / microVM isolation
HashiCorp Vault / cloud KMS	Secrets for service identities	Requests	Short-lived credentials	Dynamic secrets, envelope encryption

9.7 Observability, platform and front end

Tool / library	Purpose	Input	Output	Core algorithm
OpenTelemetry SDKs	Traces, metrics, logs across services	Spans	OTLP data	Context propagation
Langfuse	LLM traces, prompt versions, evaluations, cost	Spans, scores	Dashboards, datasets	—

Tool / library	Purpose	Input	Output	Core algorithm
Prometheus + Grafana	Platform metrics and dashboards	Metrics	Alerts, charts	Time-series storage, PromQL
DCGM exporter	GPU utilisation and memory	GPU telemetry	Metrics	—
Docker, Kubernetes, KEDA/HPA	Containers, orchestration, autoscaling	Workloads	Scaled pods	Queue- and metric-driven autoscaling
Terraform, Argo CD, GitHub Actions/GitLab CI	Infrastructure as code, GitOps, CI/CD	Code	Environments, deployments	Declarative reconciliation
Next.js / React + Tailwind	Web portal, admin console, approval UI, dashboards	API data	UI	—

10. Algorithm reference

This section gives the working definition of each algorithm the build depends on, so the application can explain itself in a demo or interview.

BM25 (lexical ranking). Scores a document by matching query terms, weighted by rarity and normalised for length. It is what makes exact identifiers and error strings findable.

$$\mathrm{BM25}\left(q,d\right) = \sum_{t \in q} \mathrm{IDF}\left(t\right) \cdot \frac{f\left(t,d\right)\ \left(k_1+1\right)}{f\left(t,d\right) + k_1\left(1-b+b\frac{|d|}{\mathrm{avgdl}}\right)}$$

with typical $k_1 \approx 1.2$ and $b \approx 0.75$.

Dense retrieval and HNSW. Queries and chunks are embedded; relevance is cosine similarity. HNSW (Hierarchical Navigable Small World) builds a layered proximity graph and greedily descends from sparse upper layers to the dense bottom layer, giving sub-linear approximate nearest-neighbour search. Its parameters are M (links per node) and ef (search breadth), which trade recall for latency.

Reciprocal rank fusion (RRF). This combines BM25, dense and graph result lists without score calibration.

$$\mathrm{RRF}\left(d\right) = \sum_{r \in R} \frac{1}{k + \mathrm{rank}_r\left(d\right)}, \quad k \approx 60$$

Cross-encoder reranking. The query and passage are encoded together, so attention sees both. This is more accurate than comparing separate embeddings, but costs one forward pass per candidate, so it is applied only to the top 50.

Contrastive embedding training (MNRL / InfoNCE). Each query is pulled toward its positive chunk and pushed from other chunks in the batch and from mined hard negatives.

$$\mathcal{L} = -\log \frac{e^{\mathrm{sim}\left(q,p^+\right)/\tau}}{e^{\mathrm{sim}\left(q,p^+\right)/\tau} + \sum_j e^{\mathrm{sim}\left(q,p_j^-\right)/\tau}}$$

Fill-in-the-middle (FIM). A training document is split into prefix, middle and suffix, and rearranged with sentinel tokens as `<PRE> prefix <SUF> suffix <MID> middle`. The model then learns to generate the middle given both sides, which is exactly the IDE completion task.

LoRA and QLoRA. The frozen weight W is updated as W′ = W + (α/r)·BA, with rank r much smaller than the weight dimensions. QLoRA stores W in 4-bit NormalFloat (quantiles of a normal distribution) with double quantisation of the scaling constants, and computes in BF16. For an 8B model with r = 16 on all linear layers, about 42M parameters (≈ 0.5%) are trained (see LEXORA v2, section 7).

Direct Preference Optimisation (DPO). DPO learns from (preferred, rejected) pairs, such as accepted versus dismissed review comments, without a separate reward model.

$$\mathcal{L}_{\mathrm{DPO}} = -\log \sigma \left(\beta \left[\log \frac{\pi_\theta \left(y_w | x \right)}{\pi_{\mathrm{ref}} \left(y_w | x \right)} - \log \frac{\pi_\theta \left(y_l | x \right)}{\pi_{\mathrm{ref}} \left(y_l | x \right)} \right] \right)$$

Knowledge distillation. A small student matches a large teacher’s softened output distribution, using a KL-divergence loss at temperature T plus the usual cross-entropy on hard labels. LEXORA uses it to create cheap classifiers and FIM draft models.

Temperature scaling (calibration). Logits are divided by a scalar T, fitted on validation data to minimise negative log-likelihood. As a result, a 0.9 confidence from L1 means about 90% correct, which lets NEXUS auto-route only high-confidence tickets. Calibration is measured with expected calibration error (ECE).

Drain (log template mining). Drain is a fixed-depth parse tree keyed by log length and leading tokens. Each new line joins the most similar template above a similarity threshold, otherwise it starts a new template. Variable fields become wildcards. This compresses millions of log lines into a few hundred templates an LLM can read.

Isolation Forest and STL. Isolation Forest isolates points with random splits; anomalies need fewer splits, so a shorter average path length means a higher anomaly score. STL decomposes a metric into seasonal, trend and residual components, and large residuals flag anomalies. Both feed the incident correlation step.

MinHash LSH (deduplication). MinHash estimates Jaccard similarity between shingle sets from signatures of minimum hash values. LSH bands the signatures so that only likely-similar pairs are compared, which makes deduplication of millions of files tractable.

Gradient-boosted trees and SHAP. LightGBM adds trees sequentially, each fitting the gradient of the loss. It uses histogram binning and leaf-wise growth for speed. TreeSHAP computes exact Shapley values per prediction, so each change-risk score comes with its top contributing factors.

PagedAttention and continuous batching. The KV cache is stored in fixed-size blocks like virtual-memory pages, which removes fragmentation. New requests join the running batch at every decoding step instead of waiting for a batch to finish. Together these give high GPU utilisation with many concurrent completions.

Prefix caching. The KV blocks for a shared prompt prefix (system prompt, file header) are reused across requests, so only the new tokens need prefill. It is the main optimisation for IDE completion.

Speculative decoding. A small draft model proposes several tokens, and the large model verifies them in one forward pass, accepting the longest correct run. The output distribution is unchanged, and latency drops when the draft model agrees often, as it does in boilerplate code.

Grammar-constrained decoding. At each step, tokens that would violate the JSON schema or query grammar are masked out of the logits. This guarantees parseable review comments, tool calls and queries.

Router scoring. Among models allowed by the policy gate, choose the one that maximises quality minus weighted cost and latency, subject to a minimum quality per task (see NEXUS v2, section 5).

Semantic cache lookup. Embed the query, find the nearest cached query within the same ACL group and corpus version, and return the cached answer if cosine similarity is at least 0.95.

11. API contracts for the application

These contracts are the minimum interface for the build. All endpoints sit behind the NEXUS gateway, require an OIDC bearer token and return a `trace_id`.

11.1 Endpoint summary

Endpoint	Method	Purpose	Main input	Main output
/v1/chat	POST (SSE)	Code, knowledge and IT chat	messages, surface, repo scope	Streamed answer with citations
/v1/complete	POST	IDE inline completion	prefix, suffix, path, language, repo	Completion text, model version
/v1/tickets/triage	POST	Classify and route a ticket	title, description, requester	Category, group, priority, confidences, duplicates
/v1/reviews	POST	Review a PR	repo, PR number or diff	Structured review comments
/v1/query/generate	POST	Text-to-query	question, target (sql, promql, logql, kql), datasource	Query, validation result, optional preview rows
/v1/incidents/{id}/brief	POST	Build incident brief	alert payload or incident ID	Brief, hypotheses, suggested actions
/v1/actions/{id}/approve	POST	Approve a proposed action	approver decision, comment	Approval token, execution status
/v1/changes/risk	POST	Score change risk	change request fields	Risk probability, SHAP factors
/v1/feedback	POST	Capture accept/reject/rating	trace_id, signal	Stored feedback ID

11.2 Example payloads

```
        // POST /v1/complete  (request)
        {
          "repo": "payments/checkout-service",
          "path": "src/checkout/validators.py",
          "language": "python",
          "prefix": "def validate_card(card: Card) -> None:\n    if not card.number:\n
",
          "suffix": "\n    return None\n",
          "max_tokens": 64
        }
        // response
        { "completion": "raise ValidationError(\"card number is required\")",
          "model": "lexora/fim-coder@2.1.0", "latency_ms": 212, "trace_id": "tr_7f3c..." }
```

```
        // POST /v1/tickets/triage  (response)
        {
          "category": {"label": "Access > VPN", "p": 0.93},
          "assignment_group": {"label": "Network-L2", "p": 0.88},
          "priority": {"label": "P3", "p": 0.81},
          "auto_route": true,
          "duplicates": [{"ticket": "INC0048211", "p": 0.91}],
          "suggested_kb": ["KB0012034"],
          "model": "lexora/triage@1.3.0", "trace_id": "tr_91ab..."
        }
```

```
        // POST /v1/reviews  (response, one comment shown)
        {
          "comments": [{
            "path": "src/checkout/service.py", "line": 118,
            "severity": "high", "category": "security",
            "message": "User input is interpolated into SQL; use parameterised queries.",
            "suggestion": "cursor.execute(\"SELECT * FROM orders WHERE id = %s\",
(order_id,))",
            "validators": {"semgrep": "confirmed", "secrets": "clean"}
          }],
          "model": "lexora/code-review@1.0.4", "trace_id": "tr_c02d..."
        }
```

```
        // POST /v1/incidents/INC-2291/brief  (response, abridged)
        {
          "impact": "Checkout error rate 7.4% (SLO 1%) since 10:42 IST",
          "hypotheses": [
            {"rank": 1, "cause": "Deploy checkout-service v3.18.0 at 10:40",
             "evidence": ["new log template E1043 ×12,480", "p99 latency +310 ms after
deploy"],
             "confidence": 0.71},
            {"rank": 2, "cause": "Payment gateway latency", "evidence": ["upstream span p99 +90
ms"], "confidence": 0.18}
          ],
          "similar_incidents": ["PM-0412", "PM-0388"],
          "proposed_action": {"id": "act_55e1", "runbook": "rollback-deployment",
                              "params": {"service": "checkout-service", "to": "v3.17.2"},
                              "requires_approval": true},
          "trace_id": "tr_4be9..."
        }
```

12. Build guide

12.1 Repository structure

```
engit-copilot/
├── lexora/
│   ├── ingest/          # connectors: git, itsm, confluence, logs
│   ├── prep/            # secrets scan, PII redaction, licence filter, dedupe, tree-sitter
chunking, Drain3
│   ├── train/           # L1–L9 training scripts and configs (Axolotl/TRL YAML)
│   ├── eval/            # pass@k harness, retrieval eval, triage metrics, RAGAS
│   └── registry/        # MLflow model-card writer (handoff contract)
├── nexus/
│   ├── gateway/         # FastAPI edge, auth, DLP + secret scan, rate limits
│   ├── router/          # policy gate (OPA client), tier scoring, fallbacks
│   ├── rag/             # indexers (code, docs, schemas), hybrid retrieval, RRF, reranker
client
│   ├── workflows/       # LangGraph graphs: helpdesk, triage, pr_review, incident, change_risk
│   ├── tools/           # MCP servers: git, itsm, idp, obs, runbook, db_readonly
│   ├── guardrails/      # LLM Guard, injection classifier, Semgrep/Checkov validators
│   └── api/             # /v1 endpoints, SSE streaming, feedback
├── clients/
│   ├── vscode-extension/   # TypeScript: completion provider, chat panel
│   ├── chat-bot/           # Slack Bolt / Teams bot with approval buttons
│   └── web-portal/         # Next.js: self-service, approvals, dashboards
├── infra/               # docker-compose (MVP), Helm charts, Terraform
└── docs/                # this specification, ADRs
```

12.2 Laptop-scale MVP stack

The MVP proves every flow end to end with small models and public data. It swaps to the production tiers by configuration only.

Concern	MVP choice	Production equivalent
Completion model	Small FIM-capable coder (≈ 1.5B) via Ollama or vLLM	L4 adapter on 1.5–7B, dedicated vLLM pool
Chat / review model	7B code instruct model, quantised	L5 adapter on ≈ 30B code model
General model	7–8B instruct, quantised	T2 self-hosted + T3 managed
Embeddings / reranker	bge-small or bge-m3 + bge-reranker-base via sentence-transformers	L2/L3 fine-tuned on TEI
Triage classifier	ModernBERT-base fine-tuned on a public IT-ticket dataset	L1 on company tickets
Vector + lexical	Qdrant + OpenSearch in Docker	Managed or clustered equivalents
Graph	Neo4j Community in Docker	Neo4j cluster
Workflows	LangGraph + Postgres checkpointer	Same, HA Postgres
Tools	Mock MCP servers for ITSM, IdP, runbooks; real GitHub API on your own repos; Prometheus + Loki in Docker with a demo app	Real enterprise systems
Observability	Langfuse + Grafana in Docker	Same, production deployment

12.3 Public data for the MVP

Need	Public source
Code search and retrieval evaluation	CodeSearchNet; your own GitHub repositories
Code generation evaluation	HumanEval, MBPP; SWE-bench (Lite) for issue-to-fix tasks
Text-to-SQL	Spider and BIRD benchmarks
Logs for Drain3 and incident demos	Loghub (LogPAI) log datasets
IT tickets for L1	Public IT service-desk / customer-support ticket datasets on Hugging Face or Kaggle (check licence)
Incident and postmortem style	Public postmortem collections published by engineering organisations (summarise, do not copy)

12.4 Build sequence (12 weeks)

Weeks	LEXORA	NEXUS	Demo-able outcome
1–2	Ingest own repos and public tickets; secrets scan, tree-sitter chunking	Gateway, auth, /v1/chat with T2; Langfuse tracing	Knowledge chat over docs with citations
3–4	L1 triage classifier; L2 embedder fine-tune	Code indexer (BM25 + dense + graph); RRF + reranker	Codebase Q&A with file/line citations; ticket triage API
5–6	L4 FIM adapter on own repos; eval harness	/v1/complete + VS Code extension with debounce/cancel	Inline completion in the IDE
7–8	L5 review adapter (SFT); L6 text-to-query	PR review workflow + Semgrep; query sandbox	Bot reviews on a real PR; NL → PromQL/SQL
9–10	L7 incident summariser; Drain3 on Loghub	Incident workflow with Prometheus/Loki and approval buttons in Slack	Alert → brief → approve rollback → verify
11–12	L8 duplicates; L9 change risk; DPO round on review feedback	Helpdesk workflow with mock Okta/ServiceNow; dashboards; load test	Full 15-minute demo across all surfaces

12.5 Demo script (15 minutes)

1. In VS Code, type in a service file and show inline completions using internal helpers. Then show the latency and model version in the trace.
2. Ask “where is card validation done and who calls it?” and show the answer with file and line citations and code-graph expansion.
3. Open a PR containing a SQL-injection bug; the bot review flags it, and Semgrep confirms it.
4. Ask “error rate for checkout in the last hour by region” and show the generated PromQL, its validation and the result.
5. Fire a demo alert; the incident brief appears in Slack with ranked hypotheses. Approve the rollback, then watch it verify.
6. As an employee, ask to reset MFA, which triggers step-up verification and the action. Then raise a VPN issue that is triaged and routed with a confidence score.
7. Paste a fake AWS key into chat to show it blocked at the gateway. Add a comment containing an injected instruction to show it ignored.
8. Close on dashboards: acceptance rate, deflection, triage accuracy, GPU use by tier and cost per accepted suggestion.

12.6 Interview drill-down map

Likely question	Where this document answers it
How do you serve code completion at 100K-employee scale?	Sections 5.1–5.3 and the prefix caching, speculative decoding and PagedAttention entries in section 10
How do you do RAG over code?	Sections 6.2–6.4 (tree-sitter chunking, code graph, hybrid retrieval)
When do you fine-tune versus retrieve?	Section 4.1
How do you stop code and secrets leaking?	Sections 4.3 and 8.1
How do agents act safely on production?	Sections 7.1, 7.4 and approval tokens
How do you evaluate a code model?	Section 4.5 (pass@k, acceptance, hallucinated-API rate)
Explain BM25, HNSW, RRF, DPO, FIM	Section 10